ForgeHTTP

Project Plan

A lightweight HTTP/1.1 web server and application framework built from scratch in modern C++, with a React admin dashboard as the frontend layer.

Goal: not to compete with Nginx — to demonstrate real command of networking, C++, OOP, concurrency, and software architecture, in a project that's finishable, deployable, and defensible in an interview.

Companion documents

ForgeHTTP_UML_Diagram.pdf — full class diagram with legend & notation
ForgeHTTP_Function_Spec.pdf — per-class, per-function specification

Table of Contents

Use this document for the why and the order. Use the two companion PDFs for the what, in detail.

1 Vision & Success Criteria

By the end of this project, running:

```
./forge --port 8080
```

starts a concurrent HTTP server that:

- Parses raw HTTP/1.1 requests over POSIX sockets (no Boost.Beast, no framework doing the real work)
- Routes requests — including path parameters like `/users/:id` — through a middleware chain
- Serves a small JSON API and static files
- Exposes a `/metrics` endpoint
- Is visualized live by a React dashboard

Definition of done

The project is successful if I can explain, unaided, in an interview:

- How `accept()` and the socket lifecycle work
- Why the thread pool needs a condition variable, and how the task queue avoids corruption
- How the HTTP parser turns raw bytes into a request object
- Why middleware is Chain of Responsibility, and why the socket wrapper uses RAII

2 Tech Stack

Concern	Choice	Why
Language	C++20	Modern features without hiding fundamentals
Build system	CMake	Industry standard, plays well with Docker
Testing	GoogleTest	Unit + integration tests
Networking	POSIX sockets (socket, bind, listen, accept, recv, send)	Forces understanding of what HTTP sits on top of
JSON (optional, later)	Small header-only lib, e.g. nlohmann/json	Only once hand-rolled JSON becomes a distraction, not a lesson
Frontend	React (+ a chart lib, e.g. Recharts)	Reuses existing React experience; talks to the C++ server's own API
Deployment	Docker + docker-compose	Designed in from day one, not bolted on

Dependencies are kept deliberately minimal. The interesting parts — sockets, HTTP parsing, concurrency, routing — are built by hand, not imported.

3 Architecture

```
CLIENT
  |
  v
TCP Socket -> HTTP Parser -> Thread Pool -> Router -> Middleware -> Controller -> Response
  |                                                                               |
  +-------------------------------------------------------------------------------+
                                  back to CLIENT
```

Each layer only knows about the layer directly below it:

- **Server** manages lifecycle only — it doesn't know how HTTP is parsed or routes are handled.
- **Socket** is a thin RAII wrapper around a file descriptor.
- **HttpParser** turns raw bytes into an HttpRequest; it knows nothing about routing.
- **Router** maps method + path to a handler; it knows nothing about sockets.
- **Middleware** chain sits between router and controller (logging, CORS, auth, rate limiting).

4 Repository Structure

```
ForgeHTTP/
+-- CMakeLists.txt
+-- README.md
+-- LICENSE
+-- include/
|   +-- server/           Server.hpp
|   +-- networking/       Socket.hpp, Connection.hpp
|   +-- http/             HttpRequest.hpp, HttpResponse.hpp, HttpParser.hpp, HttpMethod.hpp
|   +-- routing/          Router.hpp, Route.hpp
|   +-- concurrency/      ThreadPool.hpp, TaskQueue.hpp, Worker.hpp
|   +-- middleware/       Middleware.hpp, LoggerMiddleware.hpp, CorsMiddleware.hpp
|   +-- controllers/      HealthController.hpp
+-- src/                  (mirrors include/, implementation files)
+-- tests/                http/, routing/, concurrency/, integration/
+-- examples/             basic_server.cpp
+-- dashboard/            React admin dashboard (added in Milestone 9)
+-- Dockerfile
+-- docker-compose.yml
```

Not all folders are created up front — they're added as each milestone needs them.

5 Core Classes

Full method-by-method detail lives in `ForgeHTTP_Function_Spec.pdf`. This section is the short version.

Socket — RAII wrapper

```
class Socket {
public:
    Socket();
    ~Socket();               // closes the fd automatically
    void bind(int port);
    void listen(int backlog);
    Socket accept();
    std::string receive();
    void send(const std::string& data);
private:
    int fileDescriptor;
};
```

The destructor closing the fd is the concrete demonstration of RAII — no manual close() calls scattered through the codebase.

HttpRequest

Holds: method, path, version, headers, body, query parameters. Example: `GET /users?id=10 HTTP/1.1` becomes method=GET, path=/users, query["id"]="10".

HttpResponse

Holds: status code, headers, body. Provides helpers like `HttpResponse::json(...)` that generate a full HTTP response with correct Content-Type and Content-Length.

Router

```
router.get("/hello", helloHandler);
router.get("/users", getUsers);
router.post("/users", createUser);
```

Later extended to support path parameters (`/users/:id` → params["id"] = "42").

ThreadPool

```
ThreadPool pool(8);
pool.enqueue([] { handleRequest(); });
```

Built with std::thread, std::mutex, std::condition_variable, std::atomic, std::future. Replaces a naive "spawn a thread per connection" model with a fixed pool pulling work off a shared task queue.

Middleware

Chain of Responsibility: each middleware receives the request, does its work, and decides whether to call the next link.

```
Request -> Logger -> CORS -> Auth -> Rate Limiter -> Router
```

6 Design Patterns

Used deliberately, not for a checklist:

Pattern	Where	Why it's genuinely needed
RAII	Socket, locks, connections	C++-idiomatic resource safety
Factory	Building responses/handlers	Decouples construction from use
Strategy	Routing strategies (exact / prefix / parameter match)	Different route-matching algorithms, swappable
Chain of Responsibility	Middleware pipeline	Each middleware independently decides to continue or short-circuit
Observer	Server events (RequestReceived, ResponseSent, ServerStarted/Stopped)	Logger, metrics, and monitoring subscribe independently

Aiming for 4–5 patterns that solve a real problem, not ten patterns for the sake of a portfolio bullet point.

7 Observability

Structured request logging:

```
[18:42:10] GET /api/users       200  12ms
[18:42:11] GET /api/users/42    200   4ms
[18:42:12] POST /api/users      201  17ms
[18:42:15] GET /does-not-exist  404   2ms
```

A /metrics endpoint exposing:

```
{
  "requests": 12452,
  "active_connections": 17,
  "average_response_time_ms": 8.4,
  "errors": 31,
  "workers": 8
}
```

This is what elevates the pitch from “I made a server” to “I built a concurrent HTTP server with observability” — and it's what the dashboard will visualize.

8 Frontend: Admin Dashboard

A React dashboard that consumes the C++ server's own API (/metrics, request log data) — no separate backend for the dashboard itself.

Planned layout

- Header: server name/status
- Stat cards: total requests, active connections, average response time
- Requests/second chart (polling /metrics on an interval, or later via WebSocket push)
- Recent requests table (method, path, status, latency)

Stack

React + a charting library (Recharts or Chart.js), plain fetch/polling to start, no heavy framework needed. This is intentionally built **last**, once the backend actually emits real metrics to visualize — building it earlier means mocking data that later has to be rewired.

9 Deployment

Designed in from day one, not retrofitted:

- Dockerfile — builds via CMake inside an Ubuntu/Linux base image, produces the forge binary
- docker-compose.yml — runs the server (and later, the dashboard as a second service)
- Port is never hardcoded:

```
const char* port = std::getenv("PORT");
// fall back to 8080 if unset
```

10 Milestones / Roadmap

#	Milestone	Outcome
1	TCP server	Raw socket accepts a connection, sends back "Hello World"
2	HTTP parsing	Understand and parse GET/POST/PUT/DELETE, headers, body
3	Router	GET /hello, GET /users, POST /users dispatch correctly
4	Thread pool	Server becomes genuinely concurrent
5	Middleware	Logger, CORS, Auth, Rate limiting chained in front of routes
6	Static files	GET / serves HTML/CSS/JS
7	JSON API	Small real API (e.g. /api/users)
8	Metrics	/metrics endpoint live
9	React dashboard	Visualizes the running server
10	Docker + deployment	Server runs in a container, deployed somewhere reachable

Rule for the whole project: build bottom-up. Don't skip ahead to framework niceties before the TCP/HTTP fundamentals are solid — the fundamentals are the actual point of the project.

11 Immediate Next Step

Start Milestone 1:

1. Set up the repo skeleton and CMakeLists.txt
2. Write a minimal Socket class (RAII wrapper)
3. Write a minimal Server that accepts one connection and writes back a static string
4. Confirm it responds to a real browser request on localhost:8080

Only once that's solid do we move to HTTP parsing (Milestone 2).